1358. Number of Substrings Containing All Three Characters

Medium | Topics | Companies | Hint

Given a string `s` consisting only of characters *a*, *b* and *c*.

Return the number of substrings containing **at least** one occurrence of all these characters *a*, *b* and *c*.

Example 1:

> **Input:** s = "abcabc"
> **Output:** 10
> **Explanation:** The substrings containing at least one occurrence of the characters *a*, *b* and *c* *are* "abc", "abca", "abcab", "abcabc", "bca", "bcab", "bcabc", "cab", "cabc" and "abc" (**again**).

Example 2:

> **Input:** s = "aaacb"
> **Output:** 3
> **Explanation:** The substrings containing at least one occurrence of the characters *a*, *b* and *c* *are* "aaacb", "aacb" and "acb".

Example 3:

> **Input:** s = "abc"
> **Output:** 1

Constraints:

- `3 <= s.length <= 5 x 10^4`
- `s` only consists of `'a'`, `'b'` or `'c'` characters.

DS / Day-93 / 9-11-26.

1358. Number of Substrings containing all 3 characters.

We r given a string s that contains only 3 possible characters :-
- 'a' - 'b' - 'c'

We need to return the total number of substrings that contain atleast one occurrence of all 3 chars (a, b & c).

Substring ⇒ is a continuous part of a string,
Eg :- s = "abcabc"
some possible substrings are :-
'a', 'ab', 'abc', 'bca', 'cab', 'abcabc', ...
but 'ac' is not a substring bcuz its chars are not next to each other in the original string.

Ex ⇒ s = "aaacb"
O/p 3
the valid substrings are :-
1. "aaacb", "aacb", "acb"
each contains atleast a, b, & c.

Ex ⇒ s = "abc", O/p = 1.
Only "abc" contains all 3 chars.

For a string, we need to find substrings,
→ check whether each substring contains a, b & c.
→ count the valid one,

But generating every substring & checking it individually is expensive, we can avoid every substring by sliding window concept

Approach: Sliding Window means maintaining a continuous range of elements using 2 pointers,
- left = starting position of the window
- right = ending " " " "

we expand the window by moving right. Once the window contains all 3 characters, we try to shrink it by moving left.

suppose, s = "abcabc"
imagine one current window :- a b c
L R.

It contains all 3 characters, now if we move the left pointer one step to the right :-
a b c a
L R

the window still contains a, b & c, we continue shrinking while the window remains valid.

trick :- whenever our window become, valid, every substring that ends at the current right index & starts at or before left-1 is also valid.

why?

bcz removing chars from the beginning of a valid window cannot remove the chars at its end, which already contain all 3 required chars.

therefore the no. of valid substrings ending at right is :- left.

ii) simple :

1. 1st what we want to count,

consider :- s = "abcabc"

suppose our current right pointer is at index 2.

a b c a b c
0 1 2 3 4 5
L R

our current window is "abc".
does it contain all 3 characters? yes,

now, which substring ending at index 2 contain all 3 characters? only "abc".

so our answer +se, by 1.

now, let's move right to index 3, our string become,

a b c a b c
0 1 2 3 4 5
L R

our current window is "abca"

↓

it contains all 3 chars.

But, which substring ending at index 3 contain all 3 chars,

abc ✓
bca ✓
ca ✗
a ✗

(2)

thats how, we can remove the 1st char a & the remaining substring is still valid. ✓ ★

now the left pointer → one window way:- a b c a
L R

we started shrinking it,
remove the 1st a, a b c a
L R

the remaining window :- "bca".
it is still valid.

remove b a b c a
LR → ca not valid

one final positions are:-

string : a b c a
index : 0 1 2 3
L R

here left = 2;
why do we add left to the answer?
→ after shrinking, left = 2,
that means, starting at index 0 give, "abca" → valid
" " " 1 " "bca" → valid
" " " 2 " "ca" → invalid
therefore the valid starting indices are:- 0, 1.
how many? = 2, 1 left is also 2,

therefore, no. of valid substrings = left
we simply write, ans += left.

why does this work?
⇒ bcz if the window [left, right] is invalid, every shorter suffix starting at left are therefore right is also invalid.
ALL every substring starting before left contains the entire valid window that existed just before shrinking - so those substrings are all valid.

Order ::

1. expand the window using right,
2. shrink the window until it is valid.
3. once it becomes invalid, add left to the answer.

Also, the code using, ans += n - right was a slight covering variant, don't mix the 2 formulas,
- shrink until invalid, then add left,
- while valid, add n - right, then shrink.

class solution {
public:
int numberOfSubstrings(string s) {

s = "abcabc"

int n = s.length(); → n = 6.
int left = 0; → left pointer
int ans = 0; → right pointer. store the total number of valid substrings.
→ maintains a frequency of a, b & c inside our current window.

int freq[3] = {0, 0, 0}; → represents,
freq[0] = a
freq[1] = b
freq[2] = c
why
'a' - 'a' = 0
'b' - 'a' = 1
'c' - 'a' = 2.

for(int right = 0; right < n; right++) { → this loop makes our right pointer move from 0 to index 5.
freq[s[right] - 'a']++; → this tells the frequency of the character at the right pointer.
what does right do? it expands our window by including one character at a time.

for ex, right = 0;
s[right] = 'a'; we get freq['a' - 'a']++;
= freq[0]++
= freq = [1, 0, 0]
we have one a,

this checks whether our current window contains all 3 characters, freq[0] represents a, freq[1] b, so on,
all 3 conditions must be true!
[while (freq[0] > 0 && freq[1] > 0 && freq[2] > 0) {
freq[s[left] - 'a']--; → we remove the leftmost character from our current window.
any which and not if? here we want to keep shrinking the window as long as it remains valid.

for ex, if left=0;
s[left] = 'a';
then freq['a'-'a']--;
↳ freq[0]--;
we r the frequency of a char we're removing it from the window.

left++; → moves the left pointer one position forward. we have now excluded the previous leftmost char.

ans += left; adds the no. of valid substrings ending at the current right index.

return ans; → returns the total no. of valid substrings.

we maintained a fixed-size freq array only 3 elements, the variables left, right & n also required

Constant space.

time = O(n) → even though we have a for loop & a nested while loop, the complexity is not O(n²). why
→ the right pointer traverses the string once, at most n movements
→ the left pointer also moves only forward, at most n movements.
→ each char is added to & removed from the sliding window at most once.
→ frequency updates & conditional checks take constant time, O(1). So, O(n) + O(n) = O(n).

* nested loops don't always mean O(n²), when 2 pointers only move forward & each moves at most n times, the overall TC is O(n).

complete duty men:- S = "abca". index = 0 1 2 3
string = a b c a

n=4; left=0; ans=0; freq = {0,0,0};

iter-1 ⇒ right=0,
S[right] = 'a';
freq[S[right] - 'a']++;
↳ freq = [1,0,0]
now we don't have
b & c yet,
ans += left,
ans = 0 to 0

iter-2 ⇒ right=1,
S[right] = 'b'.
freq = [1,1,0];
we don't have
c yet, now,
ans += left;
ans = 0+0 = 0,

iter-3 ⇒ right=2,
S[right] = 'c';
freq = [1,1,1]
↳ window
is valid.

our current
window,
a b c a
L R
freq[S[left]-'a']--;
current to:-
left=0;
~~S[right]~~
S[left] = 'a';
remove a,
freq = [0,1,1]
move left forward,
left++;
now, left=1,
our window
a b c a
· L R
not valid
now,
ans = 0+1 = 1,

iter-4
S[right+1] = 'a';
freq = [1,1,1]
valid
window.

enter a valid loop,
remove the leftmost
one,
left = 1;
S[left] = 'b';
freq[S[left] - 'a']--;
freq = [1,0,1].
left++;
left = 2;
window = a b c a
L R
↳ not valid.
ans = 1+2 = 3.

sliding window
using two
pointers.

approach-2 Last seen index

this approach is particularly elegant, but we don't even need a sliding window with two pointers, instead, we remember the last position where each character appeared,

the main idea is maintain an array:

int last[3] = {-1, -1, -1};

it stores the most recent indices of;

index	characters	initial value
0	a	-1
1	b	-1
2	c	-1

why -1? but initially, none of the characters has appeared.

suppose:- s = "abcabc"

at index 2, we have,

a b c

0 1 2

the last seen positions are:- last[a] = 0;

last[1] = 1;

last[2] = 2;

the minimum last seen indexes:-

min(0,1,2) = 0,

what does the minimum mean? = it means the earliest position among the latest occurrences of all 3 characters.

for any substring ending at index 2:-
- starting at index 0: "abc" → valid
- " " " 1: "bc" → invalid
- " " " 2: "c" → invalid,

therefore, the no. of valid substrings ending at index 2 is:- min (last) + 1 = 0 + 1 = 1.
why +1? → bcz starting indices are 0-based,

General formula,

for every index i:
ans += min(last[0], last[1], last[2]) + 1.
if any character has not appeared yet, its last-seen index is -1, so the contribution is zero.

```
class Solution {public:
    int numberOfSubstrings(string s) {
        int last[3] = {-1, -1, -1};
        int ans = 0;
        for (int i=0; i<s.length(); i++) {
            last[s[i] - 'a'] = i;
            int curr = min({last[0], last[1], last[2]}) + 1;
        }
        return ans;
    }
};
```

→ updates the last seen position of the current char.

→ traverse the string from left to right.

→ finds the smallest last-seen index
this tells us how many valid substrings end at the current index. we add that count to the ans.

→ returns the final count.

last seen index

T.C = O(n) → one traversal of the string, with constant time operations
S.C = O(1) only 3 last seen indices and a few new chars. ___/___/___

Recommended one - we traverse the string only once
→ we don't need nested loops.
→ " " " to maintain a complicated sliding window.
→ we only stores 3 indices
→ we directly calculate the no of valid substrings ending at each position.

dry run :- s = "abcabc". indices, last = [-1, -1, -1]
ans = 0

i	s[i]	updated last[]	minimum	contribution	total ans
0	a	[0, -1, -1]	-1	0	0
1	b	[0, 1, -1]	-1	0	0
2	c	[0, 1, 2]	0	1	1
3	a	[3, 1, 2]	1	2	3
4	b	[3, 4, 2]	2	3	6
5	c	[3, 4, 5]	3	4	10

final ans = 10

Accepted 55 / 55 testcases passed

Akankshyaa submitted at Nov 09, 2026 15:59

Analysis | Solution

Save 41%

Big Diwali Dhamaka

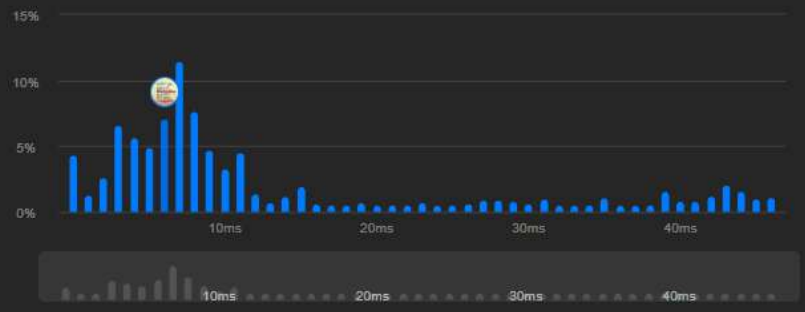
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

6 ms | Beats 74.26%

Memory

10.80 MB | Beats 93.13%



Code | C++

```
// class Solution {
// public:
//     int numberOfSubstrings(string s) {
//         int last[3]= {-1,-1,-1};
```

```
class Solution {
public:
    int numberOfSubstrings(string s) {
        int n = s.length();
        int left = 0;
        int ans = 0;
        int freq[3] = {0, 0, 0};
        for (int right = 0; right < n; right++) {
            freq[s[right] - 'a']++;
            while (freq[0] > 0 &&
                   freq[1] > 0 &&
                   freq[2] > 0) {
                freq[s[left] - 'a']--;
                left++;
            }
            ans += left;
        }
        return ans;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

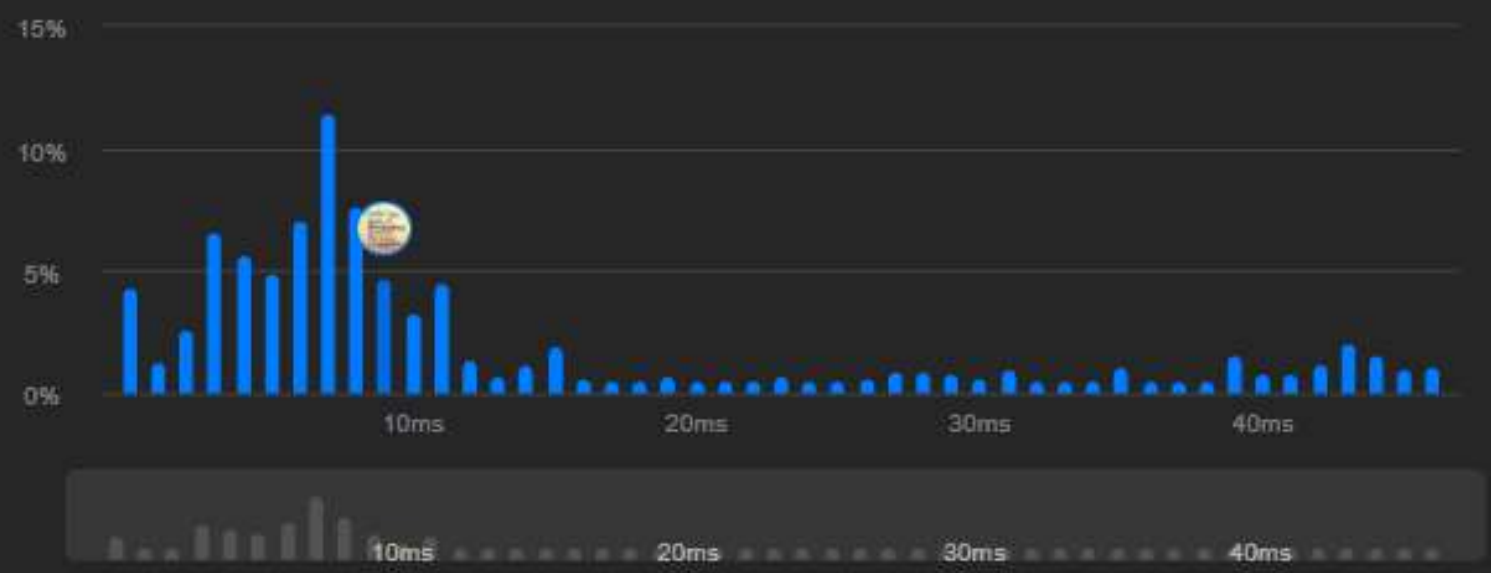
s =
"abcabc"

Runtime

9 ms | Beats **48.02%**

Memory

11.03 MB | Beats **53.05%**



Code | C++

```cpp
class Solution {
public:
    int numberOfSubstrings(string s) {
        int last[3]= {-1,-1,-1};
        int ans=0;
        for (int i=0; i<s.length();i++){
            last[s[i]-'a']=i;
            ans += min({last[0],last[1],last[2]}) +1;
        }
        return ans;
    }
};
```